SVKM'S NMIMS
School of Technology Management & Engineering
Department of Computer Engineering

AI

Artificial Intelligence

Project Report

CYBER-SWARM // Quantum Operations Hub

AI-Driven Decentralised Task Allocation for an ESP32 Micro-Robot Swarm

Submitted By:

Pranshu Sharma
SAP ID: 70022400499

Submitted To:

Dr. Gaurav Paliwal
Assistant Professor

Architecture

1. System Overview

CYBER-SWARM // Quantum Operations Hub is a real-time, browser-based multi-agent simulation and management platform for an autonomous mesh swarm of ESP32 micro-robots. Tasks appear in a spatial "tactical radar" arena; instead of a central controller assigning them, the robots compete for each task in a decentralised, Contract Net Protocol auction. Each robot calculates a bid cost using a dual AI engine, and the lowest-cost eligible robot wins. The system is live at cyberswarm.vercel.app and its source is in the public repository github.com/pranshu-create/cyber-swarm.

The platform combines six techniques: (1) a Multi-Layer Perceptron (MLP 5-6-3) bidding dispatcher with online Stochastic Gradient Descent (SGD) backpropagation; (2) a situational battery and Remaining Useful Life (RUL) model that pre-empts brownouts; (3) tabular Q-Learning for adaptive bidding policy; (4) peer-to-peer ESP-NOW mesh consensus; (5) A* pathfinding with string-pulling and smart goal projection; and (6) a live A/B benchmark dashboard that compares the AI swarm against a fixed-weight rule-based baseline.

2. Infrastructure & Deployment

- **Front end / simulator:** a single-page application (index.html) containing the arena, the neural-network visualiser, the loss-convergence chart and the benchmark dashboard. It is hosted on Vercel.
- **Node firmware:** esp32_swarm_node.ino (344 lines) runs on an ESP32 NodeMCU / WROOM-32 (Node ID 1, 2 or 3). WiFi is set to station mode and ESP-NOW so there is no master node and no router.

  Hardware hooks: battery voltage is read through a 2:1 divider on GPIO 34, wheel encoders sit on GPIO 18/19 and the motor PWM outputs on GPIO 25/26. In the repository these are documented placeholders; sensing is simulated for testing.

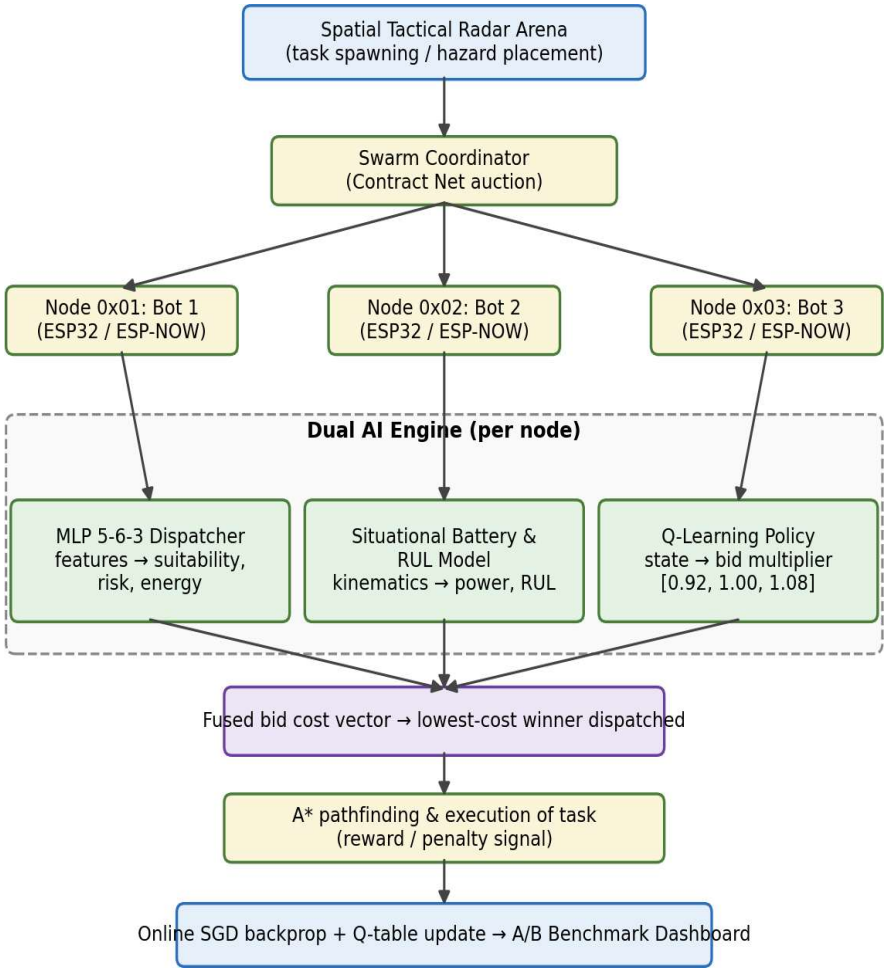


Figure 1: Cyber-Swarm system architecture and dual-AI engine

3. The Multi-Agent Auction Pipeline

Phase 1 — Task announcement. A task is spawned in the arena (priority Normal, High or Emergency). The coordinator broadcasts a `MSG_TASK_ANNOUNCE` packet to all nodes over ESP-NOW.

Phase 2 — Distributed bidding. Each node independently (a) plans an A* path to the task, (b) runs the MLP forward pass, (c) estimates RUL under the predicted workload, and (d) applies its Q-learning bid multiplier. The fused cost is broadcast as `MSG_TASK_BID`. A node whose battery is below the critical threshold (15 %) in the firmware does not bid at all.

Phase 3 — Brownout gate and winner selection. If a node's feasibility ratio (RUL / estimated mission time) is below 1.15, its bid is rejected pre-emptively and the task is re-auctioned. Among eligible bids the lowest cost wins; the winner announces `MSG_WINNER_CLAIM`. If a node later fails mid-task, `MSG_TASK_DROPPED` triggers an immediate re-auction.

Phase 4 — Execution and learning. The winner navigates (A* with string-pulling), services the task, and receives a reward or penalty. That signal drives an online SGD update of the MLP weights and a Bellman update of the Q-table, and telemetry feeds the A/B dashboard.

4. AI Components in Detail

4.1 Deep Neural Network Dispatcher (MLP 5-6-3)

A feed-forward network replaces a naive linear weighting. The input vector $X \in \mathbb{R}^5$ holds normalised features:

Input	Feature	Meaning
x_0	$d_{path} / d_{arena_diag}$	Normalised A* path distance
x_1	Battery / 100	State of charge in [0, 1]
x_2	ρ_{hazard}	Local hazard (wall) density
x_3	Urgency	Normal = 0.4, High = 0.75, Emergency = 1.0
x_4	Δ_{swarm}	Swarm spatial dispersion index

Hidden layer (6 units, tanh) and output layer (3 units, sigmoid):

$$z^{(1)} = W^{(1)}X + b^{(1)},\ H = \tanh(z^{(1)});\ z^{(2)} = W^{(2)}H + b^{(2)},\ Y = \sigma(z^{(2)}) = 1 / (1 + e^{-z(2)})$$

- y_0 **Dispatch Suitability** (higher suitability lowers the bid cost).
- y_1 **Mission Failure Risk** (raises the bid cost when entrapment or failure is likely).
- y_2 **Energy Efficiency** (rewards energy-conserving routes).

Training is online. After each task the loss is $L = (1 / 6)\,\Sigma\,(y_i - y_{target,i})^2$ and the gradients are:

$$\delta^{(2)} = (Y - Y_{target}) \odot Y \odot (1 - Y);\ \delta^{(1)} = ((W^{(2)})^T\,\delta^{(2)}) \odot (1 - H^2);\ W \leftarrow W - \eta\,\delta\,(input)^T,\ \eta = 0.08$$

4.2 Situational Battery & Remaining Useful Life (RUL) Model

Rather than a constant percentage drain, power draw depends on how the robot is actually moving:

$$P_{sit} = P_{idle} + k_v \cdot v^2 + k_\omega \cdot |d\theta/dt| + k_{detour} \cdot \rho_{hazard}$$

Here P_{idle} = 0.5 W is standby draw; the v^2 term models propulsion and drag (35 W baseline); the turning term adds up to +6.5 W for steering torque; and the detour term adds up to +4.2 W for micro-corrections in obstacle corridors. The estimator and gate are:

$$RUL = (SoC \cdot Capacity_{Wh} / \text{mean } P_{sit}) \times 3600\ s;\ \text{Feasibility Ratio} = RUL / T_{est_mission};\ \text{reject if ratio} < 1.15$$

A feasibility ratio below 1.15 means less than a 15 % safety margin. The model flags a critical brownout hazard, refuses the assignment and re-auctions it to healthier nodes.

4.3 Reinforcement Learning (Q-Learning) Bid Policy

Element	Definition
State space	Battery {LOW < 35 %, MED 35–70 %, HIGH > 70 %} × Distance {NEAR < 5 m, MED 5–12 m, FAR > 12 m} × Queue {EMPTY 0, LIGHT 1, HEAVY ≥ 2} = 27 discrete states
Action space	Bid multipliers [0.92, 1.00, 1.08]: AGGRESSIVE (under-bid), MODERATE, CONSERVATIVE (save power)
Update rule	$Q(s,a) \leftarrow Q(s,a) + \alpha\,[\,R + \gamma \max_{a'} Q(s',a') - Q(s,a)\,]$
Hyper-parameters	$\alpha = 0.20$, $\gamma = 0.90$, ϵ decays from 0.30 to a floor of 0.05

4.4 Node-Level Bid Function in the ESP32 Firmware

The firmware implements a deliberately simple, shared baseline cost function that matches the JavaScript simulation: cost = 0.5 · (normalised distance) + 0.3 · (1 − battery / 100) + 0.2 · (busy ? 1 : 0). Lower is better. The auction is masterless: every node buffers the bids it hears for 100 ms, then evaluates locally and the node with the lowest cost claims the task. This is the fixed-weight baseline against which the AI engine is benchmarked in Section 6.

About the Evaluation Data

Unlike a recorded dataset, Cyber-Swarm produces its evidence through a built-in, one-click A/B benchmark (`runBenchmarkTrial()`). It runs the same stress scenario twice, once with the fixed-weight rule-based dispatcher and once with the full AI suite, and reports four metrics. The project documentation reports results from 5-task stress trials.

Metric	Rule-based baseline	Cyber-Swarm AI	Reported improvement
Average task makespan	14.7 s	8.9 s	39.4 % faster
Energy consumption per task	2.78 Wh	1.72 Wh	38.2 % conserved
Task drop rate under crisis	18.5 %	0.0 %	Zero task loss
Fleet load-balance entropy	0.48	0.94	More even distribution

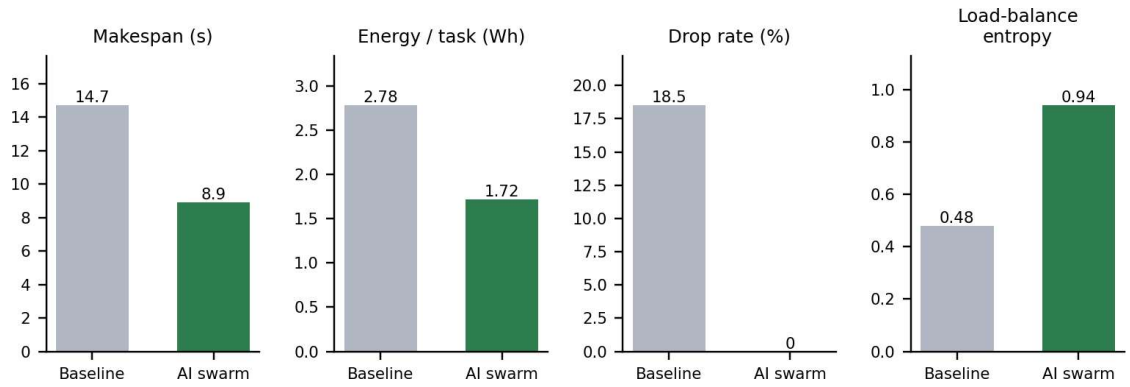


Figure 2: A/B benchmark — baseline versus AI swarm (values from the project documentation)

Reading the numbers. The two headline efficiency gains are consistent with the table: (14.7 − 8.9) / 14.7 ≈ 39.5 % and (2.78 − 1.72) / 2.78 ≈ 38.1 %. The small difference from the reported 39.4 % and 38.2 % is attributable to rounding of the underlying measurements. Load-balance entropy approaches 1 as work spreads evenly, so a move from 0.48 to 0.94 means the baseline concentrates tasks on a few robots while the AI swarm shares them.

Key source files.

File / symbol	Purpose
index.html	Single-page app: styles, arena, neural canvas and markup
SwarmNeuralNetwork	MLP 5-6-3 forward inference and SGD backpropagation
computeSituationalPower()	Kinematic power and turning-torque calculation
computeBotRUL()	Continuous Remaining-Useful-Life predictor
calculateBid()	Fuses neural suitability, brownout pre-emption and Q-action
runBenchmarkTrial()	One-click A/B comparison runner
exportVivaReport()	Generates a downloadable evaluation report
esp32_swarm_node.ino	Decentralised ESP-NOW node firmware (auction protocol)

PEAS Analysis

1. Top-Level System PEAS (Swarm Coordinator / Auction Layer)

This is the workflow engine that announces tasks, collects bids and enforces the safety gate.

Performance Measure

- Minimum average task makespan and energy per task.
- Zero task drop rate, achieved through brownout pre-emption and re-auction.
- Balanced load across the fleet (load-balance entropy close to 1).
- Correct masterless consensus: exactly one winner per task, the lowest-cost eligible bidder.

Environment

- A bounded 2-D arena with walls, hazard regions and dynamically spawned tasks of varying urgency.
- A three-node ESP32 swarm communicating over a broadcast ESP-NOW mesh.
- Web dashboard (Vercel) for visualisation, benchmarking and report export.

Actuators

- ESP-NOW broadcast of task announcements, bids, winner claims and drop notices.
- Task dispatch and re-auction triggers.
- Motor PWM outputs for locomotion (GPIO 25/26 on hardware).
- Dashboard renderers (neural visualiser, loss chart, benchmark panel).

Sensors

- Battery voltage via ADC (GPIO 34, 2:1 divider; LiPo 3.2–4.2 V mapped to 0–100 %).
- Wheel encoders (GPIO 18/19) for odometry.
- Incoming ESP-NOW packets from peers; task and hazard events from the arena.

2. Component-Specific PEAS Analyses

2.1 MLP Bidding Dispatcher

P	Low MSE loss; bids that rank robots in the true best-to-worst order; fast inference on a 5-input network.
E	Normalised feature vectors from the arena and fleet state; reward/target signals after each task.
A	Outputs suitability, failure-risk and energy ratings that adjust the bid cost; SGD weight updates ($\eta = 0.08$).
S	Path distance, battery SoC, hazard density, urgency, swarm dispersion.

2.2 Situational Battery / RUL Model

P	Accuracy of RUL prediction; 0 % in-transit brownouts; few false rejections of healthy nodes.
E	Robot kinematics (speed, turn rate) and obstacle density along the planned path.
A	Accept or reject a bid (feasibility ratio against the 1.15 threshold); trigger re-auction.
S	State of charge, velocity, angular rate, hazard density, estimated mission time.

2.3 Q-Learning Bid-Policy Agent

P	Cumulative reward over tasks; convergence of the Q-table; sensible exploration-to-exploitation decay.
E	A 27-state discretised world of battery, distance and queue load.
A	Choose a bid multiplier of 0.92, 1.00 or 1.08 (ϵ-greedy).
S	Discretised battery band, distance band and queue length; the reward signal.

2.4 A* Navigation Agent

P	Shortest collision-free path; path smoothness; low detour cost around hazards.
E	Grid map of the arena with walls and hazard cells.
A	Waypoint list after string-pulling; smart projection of unreachable goals to the nearest free cell.
S	Map occupancy, robot pose, goal coordinates.

2.5 ESP32 Swarm Node (Firmware)

P	Reliable packet exchange; correct local winner determination; timely re-auction on failure.
E	Peer ESP-NOW mesh, a 100 ms bidding window, three-node network.
A	Broadcast packets (announce, bid, claim, drop); update of local node status (IDLE, MOVING, WORKING, CRITICAL).
S	ADC battery reading, encoder ticks, received `SwarmPacket` structures.

BFS / DFS Approach

The diagram shows the life of one task in Cyber-Swarm. A task spawns and is announced to all nodes; each of the three bots computes its bid in parallel; a join gate (the brownout check, which needs every bid) either rejects a bid and loops back to the auction or allows the Contract Net winner to be chosen. The winner then navigates, executes, receives a reward signal, and the learning updates feed the telemetry dashboard. The red dashed edge is the re-auction back-edge to the auction node A.

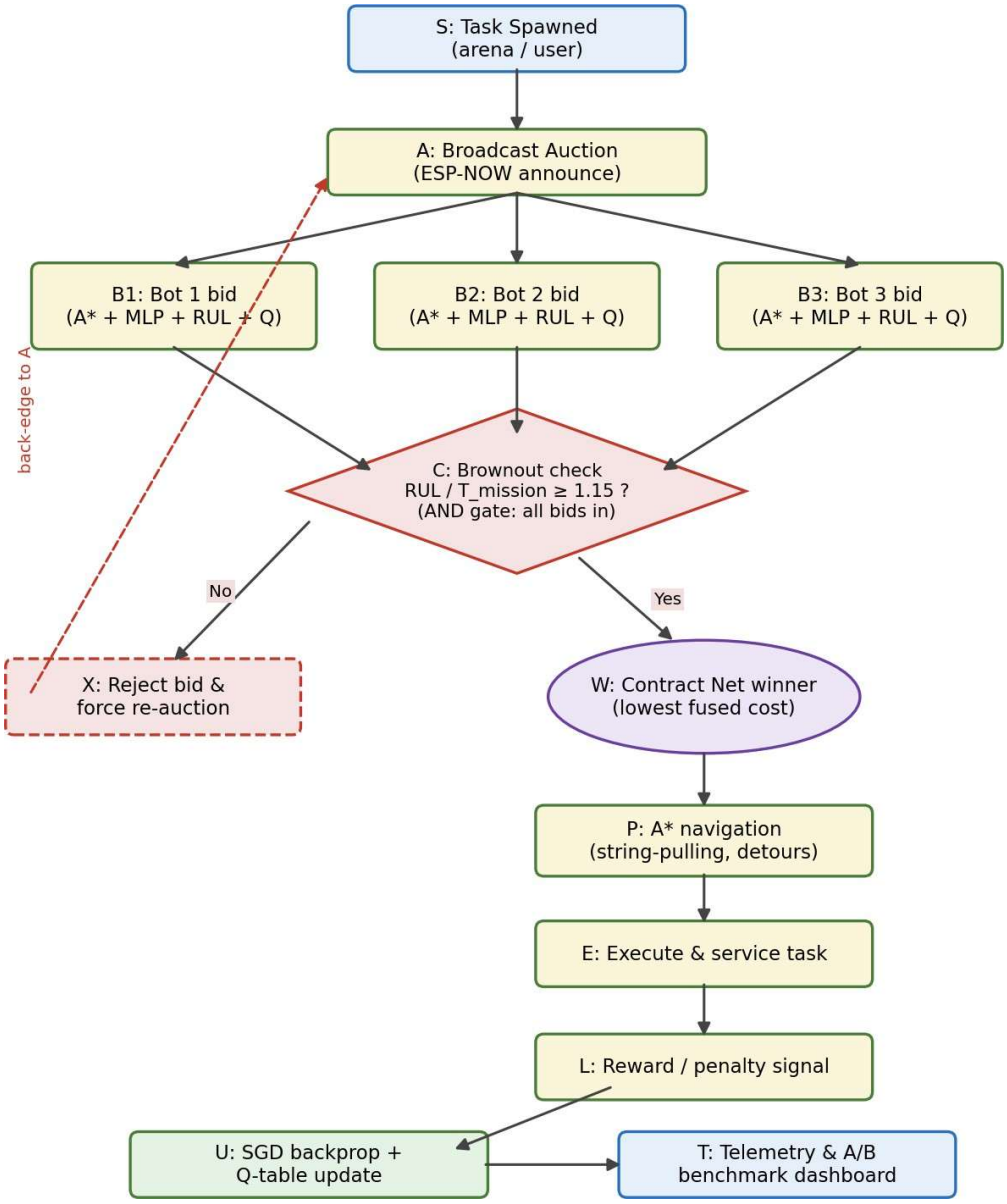


Figure 3: Task-allocation process flow used for BFS and DFS traversal

1. Breadth-First Search (BFS)

BFS explores the diagram level by level, completing every parallel branch at a depth before descending. Using a queue, the visit order is below; the back-edge from X to A is ignored because A has already been visited, which is how BFS avoids infinite re-auction cycles.

- **Level 0:** S — Task spawned
- **Level 1:** A — Broadcast auction
- **Level 2:** B1, B2, B3 — Bot 1, Bot 2 and Bot 3 bid computation (parallel)
- **Level 3:** C — Brownout feasibility check (AND gate)
- **Level 4:** X — Reject and re-auction; W — Contract Net winner
- **Level 5:** P — A* navigation

- **Level 6:** E — Execute task
- **Level 7:** L — Reward / penalty signal
- **Level 8:** U — SGD + Q-table update
- **Level 9:** T — Telemetry and benchmark dashboard

Full BFS order: S → A → B1 → B2 → B3 → C → X → W → P → E → L → U → T (13 nodes, time complexity O(V + E)). BFS reflects reality well here: the three bots really do bid simultaneously, so exploring them as one level matches the parallel auction.

2. Logical Depth-First Search (DFS)

A logical DFS dives as deep as possible along one branch. At the join gate C it cannot proceed until all three bids exist, so it pauses, backtracks to the auction node and resolves the missing branches, then resumes depth traversal once the gate condition is met.

- 1. S — Task spawned
- 2. A — Broadcast auction
- 3. B1 — Bot 1 bid computation
- 4. C — Brownout check (logical block: bids from Bot 2 and Bot 3 are missing; pause and backtrack to A)
- 5. B2 — Bot 2 bid computation (backtrack)
- 6. B3 — Bot 3 bid computation (backtrack)
- 7. C — Brownout check (condition met: all bids present; depth traversal resumes)
- 8. X — Reject and re-auction (its edge returns to A, already visited; backtrack)
- 9. W — Contract Net winner
- 10. P — A* navigation
- 11. E — Execute task
- 12. L — Reward / penalty signal
- 13. U — SGD + Q-table update
- 14. T — Telemetry and benchmark dashboard

Comparison. BFS finds the shallowest route to every node and captures the parallel bidding naturally; DFS best expresses the sequential life of a single winning task and highlights the synchronisation constraint at C. Both visit every node exactly once, so both run in O(V + E).

Hill Climbing Approach

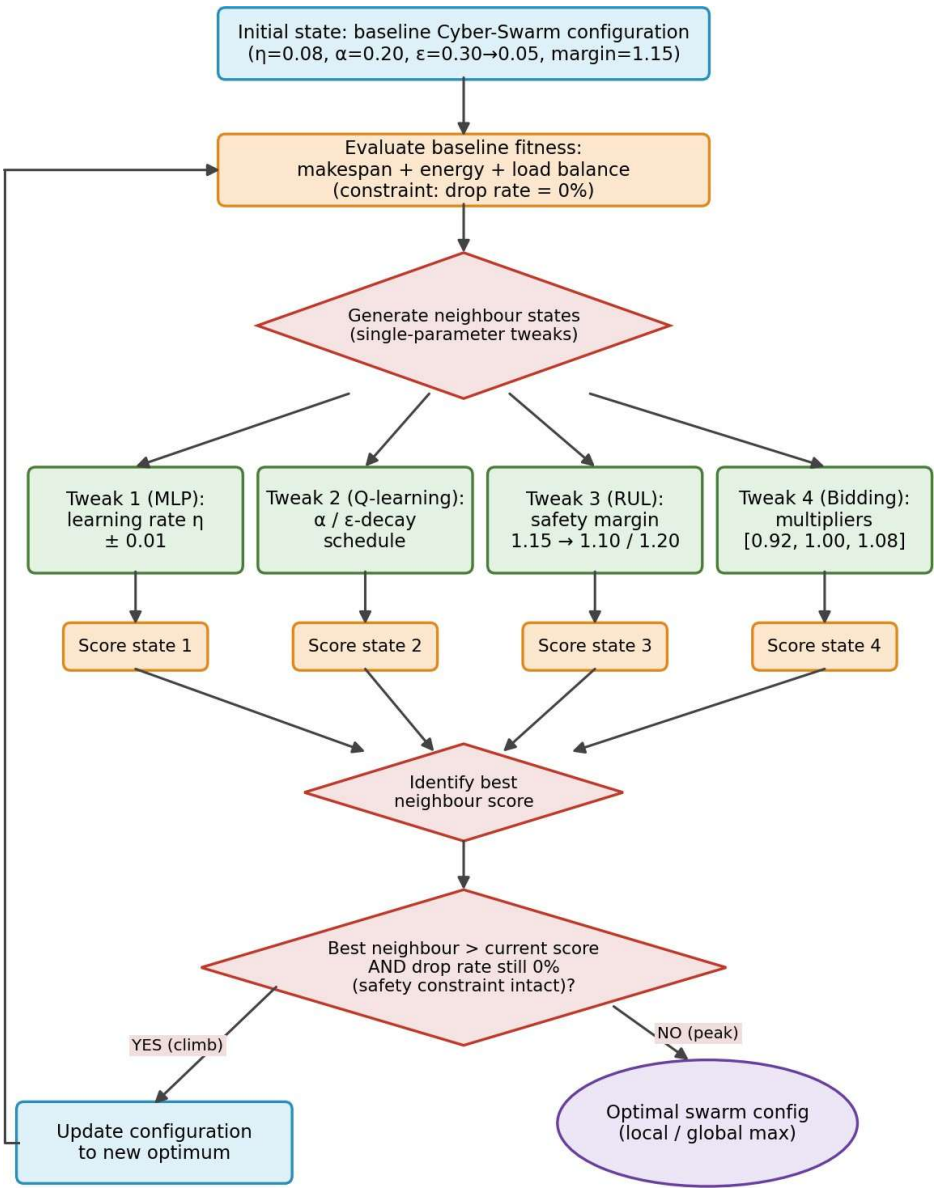


Figure 4: Steepest-ascent hill climbing applied to the swarm's tunable parameters

The flowchart shows a Steepest-Ascent Hill Climbing procedure for tuning Cyber-Swarm. It is a proposed optimisation framework for the configuration that the project exposes, rather than a feature of the repository.

1. Initialisation and Baseline Evaluation

The state is the current configuration vector: MLP learning rate η = 0.08, Q-learning α = 0.20 and ϵ schedule 0.30 → 0.05, brownout safety margin 1.15 and bid multipliers [0.92, 1.00, 1.08]. Fitness is evaluated with the built-in benchmark as a weighted combination of makespan (lower is better), energy per task (lower), and load-balance entropy (higher), subject to a hard constraint that the task drop rate stays at 0 %.

2. Generating Neighbour States

- **MLP dispatcher:** nudge the learning rate η up or down by 0.01 to trade learning speed against stability.
- **Q-learning agent:** adjust α or the ϵ-decay schedule to change how quickly the policy settles.
- **RUL model:** move the safety margin from 1.15 to 1.10 or 1.20 to trade completed missions against false rejections.
- **Bidding profile:** alter the spread of the multipliers (for example 0.90 / 1.00 / 1.10).

3. Scoring and the Constraint Check

Each neighbour (state 1 to 4) is scored with the same benchmark scenario. The best neighbour is adopted only if it improves the fitness and keeps the drop rate at 0 %. This mirrors a standard safety-first design: efficiency gains are never allowed to compromise reliability, for instance by lowering the safety margin so far that brownouts return.

4. Climb or Terminate

- **YES (climb):** update the configuration and re-run the evaluation loop from the new state.
- **NO (peak reached):** no single-parameter tweak improves fitness without violating the constraint, so the algorithm halts at a local or global maximum.

Limitations of hill climbing. Because the landscape of swarm hyper-parameters is noisy and has interacting parameters, plain steepest ascent can stall on a local maximum or plateau. Common remedies are random restarts, simulated annealing, or averaging several benchmark runs per neighbour to reduce evaluation noise.

Limitations, Future Work and Conclusion

1. Current Limitations

- The headline results come from a simulator and short 5-task stress trials; larger, repeated trials with confidence intervals would make the claims more robust.
- The reported 0 % drop rate and the 100 % zero-loss figure apply to the tested scenarios and should not be read as a general guarantee.
- The ESP32 firmware currently implements the linear baseline bid function. The MLP, RUL and Q-learning logic are implemented in the browser simulation and are not yet ported to the microcontroller.
- Sensor hooks (battery ADC, encoders) are placeholders on hardware; battery drain and motion are simulated.
- ESP-NOW broadcast is unencrypted in the current setup (`peerInfo.encrypt = false`) and uses a fixed 100 ms bidding window, so a late bid is ignored.

2. Future Work

- Port a quantized MLP and the RUL estimator to the ESP32 (for example fixed-point inference) so the AI bid runs on-device.
- Run physical three-robot trials with real battery and odometry data to calibrate the power-model coefficients.
- Use hill climbing or Bayesian optimisation to tune the hyper-parameters, as outlined in Section 5.
- Enable ESP-NOW encryption and add packet sequence numbers to defend against spoofed or replayed bids.

3. Conclusion

Cyber-Swarm shows how three complementary AI techniques can be combined to solve a decentralised allocation problem: a neural network that scores task suitability, a physics-informed battery model that prevents mission failure, and reinforcement learning that adapts bidding behaviour. In the project's own A/B benchmark the AI swarm completed tasks about 39 % faster, used about 38 % less energy per task, eliminated dropped tasks in the tested crisis scenario and distributed load far more evenly than the rule-based baseline. The BFS/DFS and hill-climbing analyses further describe how the allocation workflow can be traversed and how its parameters could be optimised without sacrificing its safety constraint.

References

- Project repository: github.com/pranshu-create/cyber-swarm (PROJECT_DOCUMENTATION.md, index.html, esp32_swarm_node.ino).
- Live demo: cyberswarm.vercel.app.
- R. G. Smith, "The Contract Net Protocol", IEEE Transactions on Computers, 1980.
- C. Watkins and P. Dayan, "Q-learning", Machine Learning, 8, 1992.
- P. Hart, N. Nilsson and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths", 1968 (A*).